CS 4476 PS4

Jinseo Lee
Jlee4223@gatech.edu
903950086

Part 1: Standard Scaler: Why did we use StandardScaler instead of looping over all the dataset twice for mean and standard deviation? Why a simple loop will not be a good choice in a deployed production grade ML system?

Standard Scaler is memory efficient. It can process data in batches instead of loading the entire, massive dataset into memory, which is essential for production. It's also highly optimized, often computing mean and standard deviation in a single pass.

Part 1: Why do we normalize our data (0 mean, unit standard deviation)?

Normalization speeds up and stabilizes training. A zero mean prevents inefficient, 'zig-zagging' optimization, while a unit standard deviation stops activation functions from 'saturating', which causes vanishing gradients and stops learning.

Part 3: Loss function. Why did we need a loss function?

A loss function is essential because it quantifies the model's error. It calculates a single number (loss) that measures how far the model's prediction is from the correct, ground-truth label. This loss value is the objective that the optimizer tries to minimize. Without a loss function, the model has no way to know if it's getting better or worse, and it cannot learn.

Part 3: [2 pt] KL-divergence is defined as

$$D_{KL}(P||Q) = \mathbb{E}_{X\sim P}\left[\log\frac{P(x)}{Q(x)}\right]$$

Cross-entropy is defined as

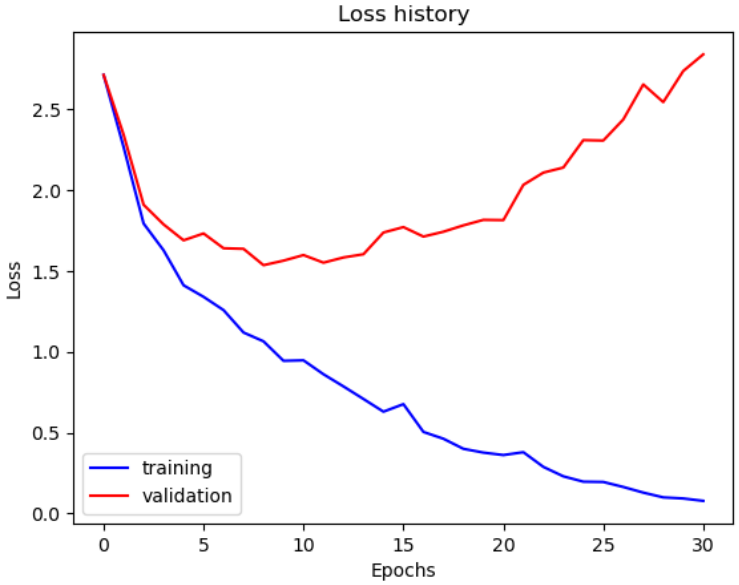
$$H(P,Q) = -\mathbb{E}_{X\sim P}\log Q(x)$$

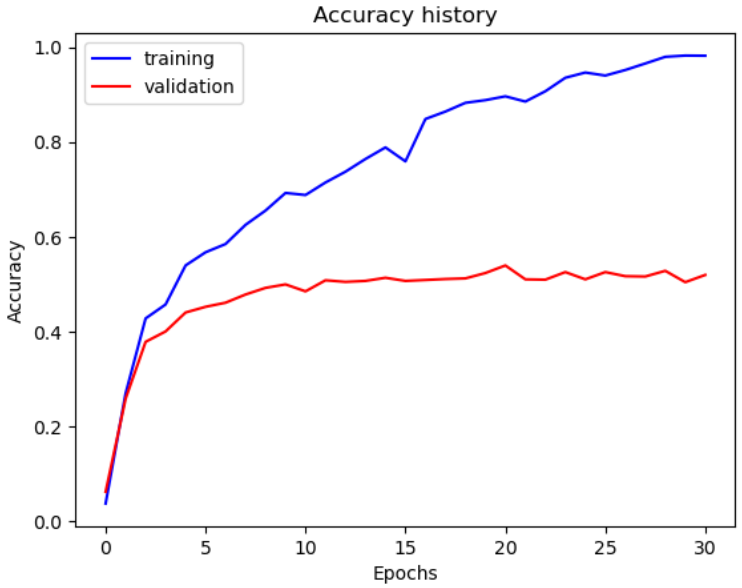
How would you write cross-entropy in terms of KL-divergence

Cross-entropy is the sum of the KL-divergence and the entropy of the true distribution, P

$$H(P,Q) = D_{KL}(P||Q) + H(P)$$

Part 5: Training SimpleNet





Final training accuracy value: 0.9822

Final validation accuracy value: 0.5193

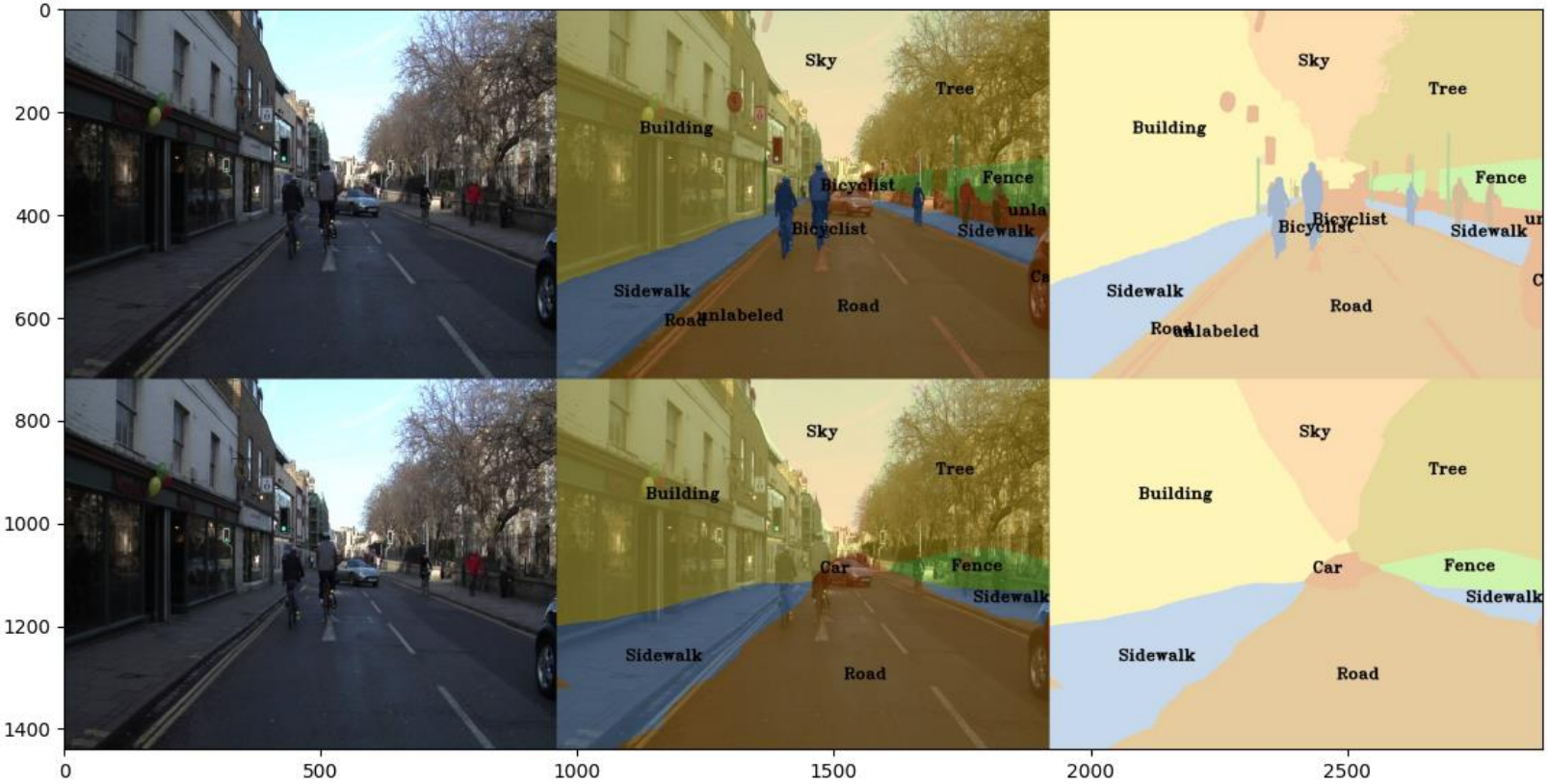
Part 6: Simple Segmentation Net

Class Index	Class name	Simple Segmentation Net Class IoU
0	Building	0.8662
1	Tree	0.8548
2	Sky	0.8955
3	Car	0.6875
4	SignSymbol	0.0000
5	Road	0.9088
6	Pedestrian	0.1294
7	Fence	0.5053
8	Column_Pole	0.0002
9	Sidewalk	0.7652
10	Bicyclist	0.0112

Validation mIoU: 0.5113

Number of Epochs: _50___

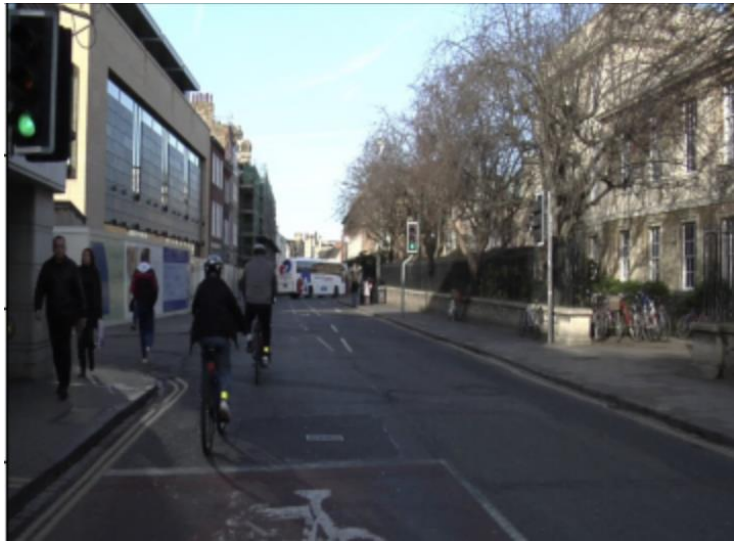
Part 6: Simple Segmentation Net



round

Part 6: Simple Segmentation Net

Which classes have the lowest mIoU? Why might they be the most difficult? Provide an example RGB image from Camvid that illustrates your point.



Based on the results, the classes with the lowest IoU were SignSymbol (0.0000) and Column_Pole (0.0002).

These classes are the most difficult because they represent objects that are either very small or very thin. The model is a simple baseline that aggressively downsamples the image to a tiny 7x7 feature map, which loses all fine-grained detail. This low resolution makes it nearly impossible to distinguish small, thin objects. The example image illustrates this perfectly, as it contains multiple Pedestrians, Bicyclists, and Column_Poles that are likely "eaten" or averaged out by the surrounding Road or Building pixels during the downsampling process.

Conclusion: briefly discuss what you have learned from this project.

In this project, I learned how to build, train, and debug deep learning models in PyTorch. I started with a simple CNN for scene classification, where I learned how to diagnose and fix overfitting using data augmentation and dropout. Then, I moved on to a more complex semantic segmentation task, implementing a simple baseline and then the advanced PSPNet. This taught me powerful techniques like dilated convolutions to gain a wider receptive field and pyramid pooling to capture context at multiple scales.

Theory: Neural Nets

1. Given are the layers of a neural network. Calculate the total number of learnable parameters for each layer and in total.

```
(conv1): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))
(maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0)
(conv2): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1))
(maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0)
(fc1): Linear(in_features=3200, out_features=10, bias=True)
(softmax): Softmax(dim=1)
```

a) Conv 1: 4864
b) Conv 2: 73856
c) FC1: 32010
d) Total (sum all 3): 110730

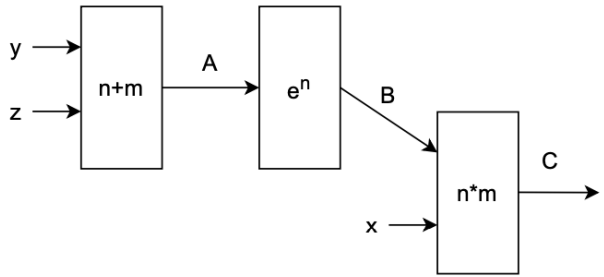
Note: You can look to pytorch docs to understand what each of the parameters correspond to.

Theory: Neural Nets (cont.)

Here is the computational graph for the function $f(x, y, z) = x * e^{y+z}$. We want to compute the partial derivative of this function with respect to each input. Here are the three intermediate functions

$A = y + z$ $\quad$ $B = e^A$ $\quad$ $C = x * B$

From these we build the feed forward computation graph, with inputs n, m. The top input is n and the bottom is m.



a. What is the correct expression for $\frac{\partial C}{\partial x}$?

$C = xB \rightarrow \frac{\partial C}{\partial x}$ = B

b. What is the correct expression for $\frac{\partial C}{\partial B}$?

$$C = xB \rightarrow \frac{\partial C}{\partial B} = x$$

c. What is the correct expression for $\frac{\partial B}{\partial A}$?

$$e^A$$

d. What is the correct expression for $\frac{\partial A}{\partial y}$?

$$1$$

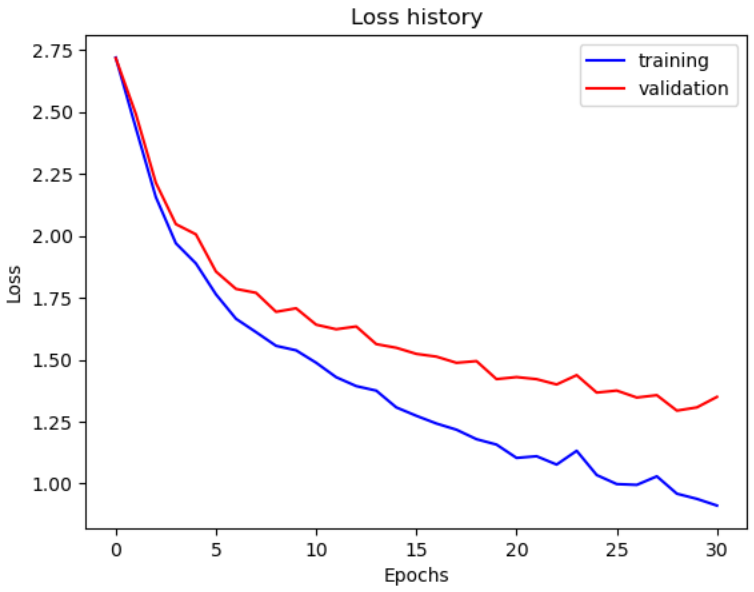
e. What is the correct expression for $\frac{\partial C}{\partial y}$?

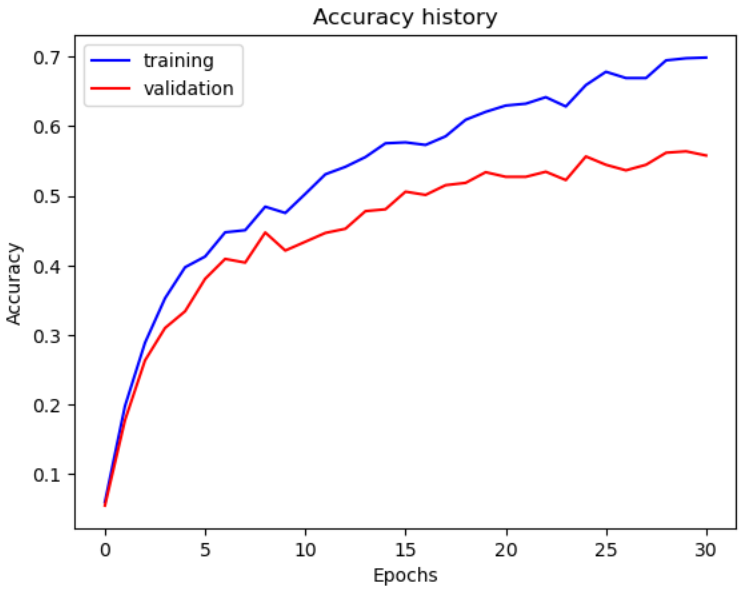
$\frac{\partial C}{\partial y} = \frac{\partial C}{\partial B} * \frac{\partial B}{\partial A} * \frac{\partial A}{\partial y}$ = x e^A

EC1.1: Screenshot of your get_data_augmentation_transforms()

```python
def get_data_augmentation_transforms(
    inp_size: Tuple[int, int], pixel_mean: np.array, pixel_std: np.array
) -> transforms.Compose:
    """
    Returns the data augmentation + core transforms needed to be applied on the train set. Put data augmentation transforms before code transforms.

    Note: You can use transforms directly from torchvision.transforms

    Suggestions: Jittering, Flipping, Cropping, Rotating.

    Args:
    - inp_size: tuple denoting the dimensions for input to the model
    - pixel_mean: the mean of the raw dataset
    - pixel_std: the standard deviation of the raw dataset
    Returns:
    - aug_transforms: transforms.compose with all the transforms
    """

    #comment out the raise error when you start writing code
    # raise NotImplementedError('get_data_augmentation_transforms not implemented')

    return transforms.Compose(
        [
            ###########################################################################
            # Student code begin
            ###########################################################################
            transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1),
            transforms.RandomHorizontalFlip(),
            transforms.Resize(inp_size),
            transforms.Grayscale(num_output_channels=1),
            transforms.ToTensor(),
            transforms.Normalize(mean=pixel_mean[0], std=pixel_std[0])
            ###########################################################################
            # Student code end
            ###########################################################################
        ]
    )
```

EC1: Training to solve overfitting





Final training accuracy value: 0.6988

Final validation accuracy value: 0.5580

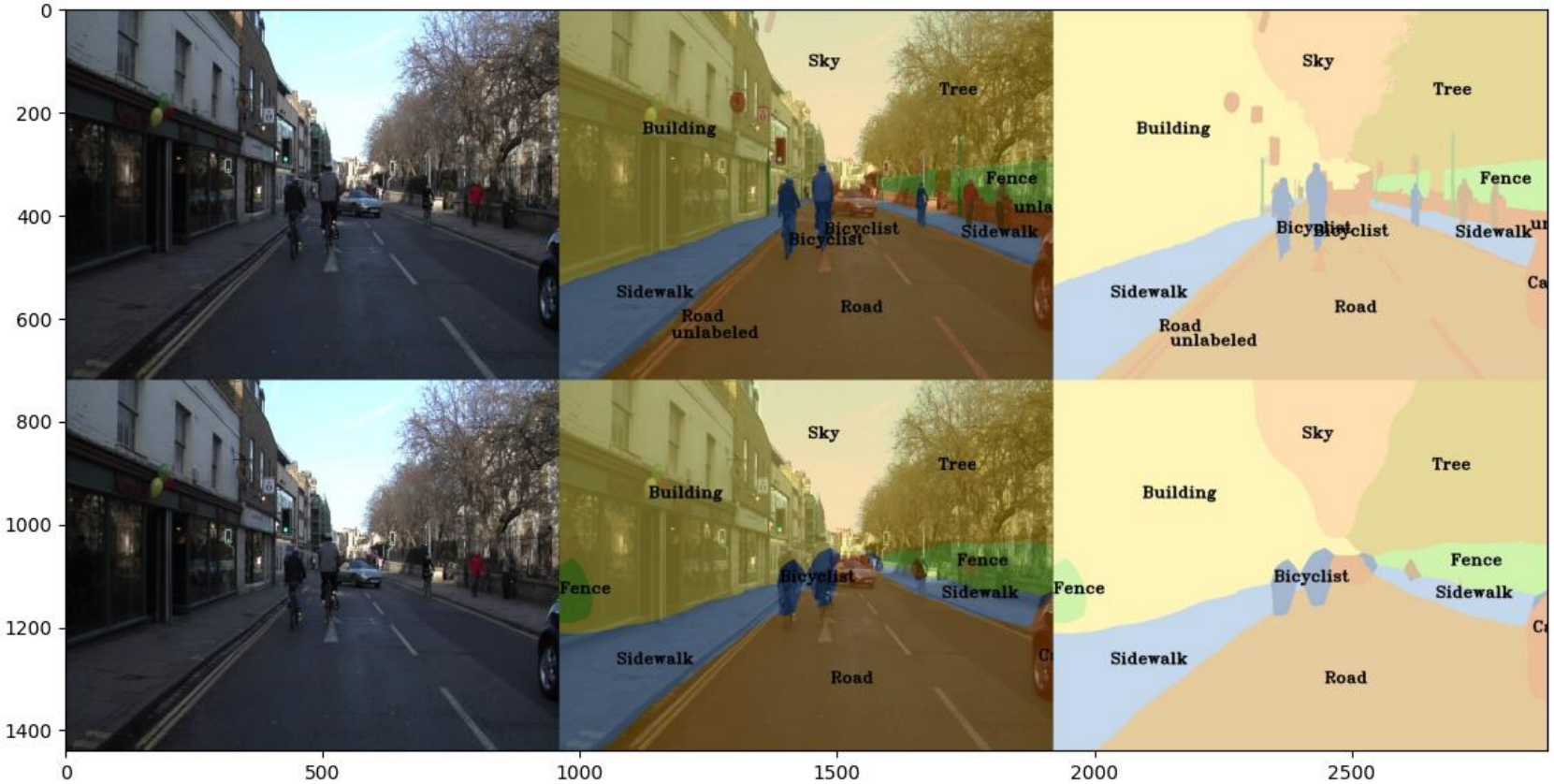
EC2: PSPNet

Class Index	Class name	PSPNet Class IoU
0	Building	0.8988
1	Tree	0.8820
2	Sky	0.9228
3	Car	0.8543
4	SignSymbol	0.0000
5	Road	0.9428
6	Pedestrian	0.3126
7	Fence	0.6272
8	Column_Pole	0.0017
9	Sidewalk	0.8316
10	Bicyclist	0.6458

Validation mIoU: 0.6291

Number of Epochs: __50__

EC2: PSPNet



und